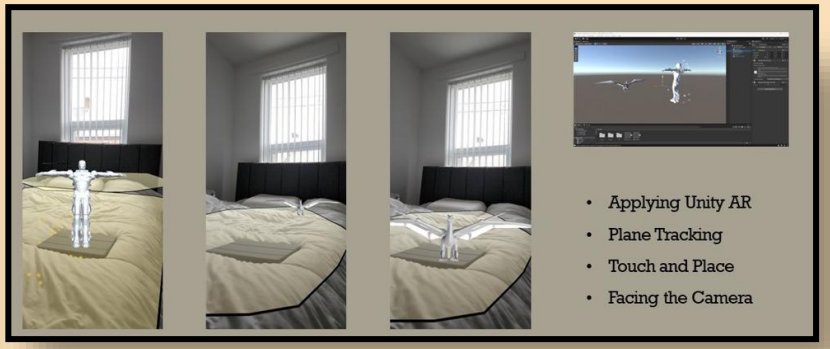
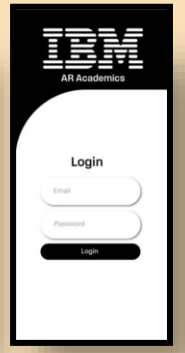
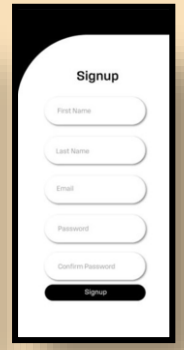
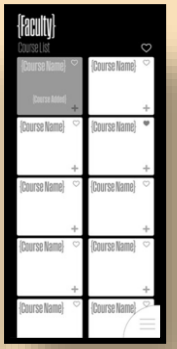
‹ Location-Based Educational AR App ›

Language: C#, Unity
Role: A team project with IBM as the client to build a location-based educational app for the campus. Inspired by Pokémon GO, proposed placing and interacting with 3D objects in the real world. Using Unity's AR framework, implemented plane tracking to anchor 3D objects on flat surfaces, touch-to-place interaction and objects that turn to face the camera. Tapping an object shows pop-up information on nearby campus spots.



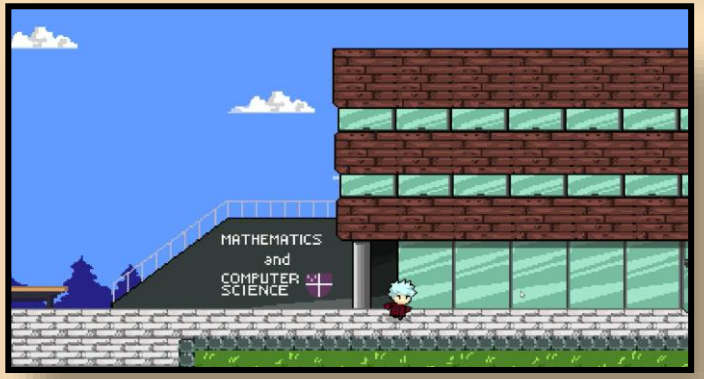






‹ University Campus RPG Game ›

Language: C#, Unity
Role: A game development assignment with two requirements: a university campus setting and a museum theme. Drawing on Night at the Museum, created a game in which campus statues come to life and chase the player by tracking their coordinates, while the player collects weapons on display to defeat enemies and escape. Used Machinations to balance enemy spawn timing and difficulty, and built a loot system for collecting and swapping weapons, each with its own attack power, range and knockback. Characters and animations used SPUM assets, while backgrounds, floor tiles and weapons were hand-drawn in Aseprite.



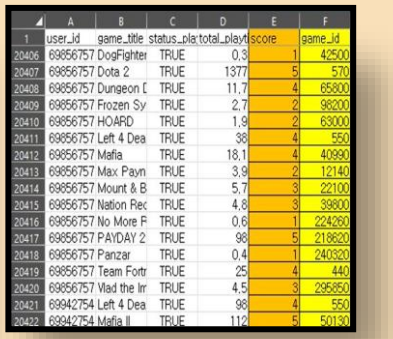


For a sense of depth, added a delayed camera follow to character movement and a parallax background of layers moving at different speeds. Settings include language, volume and a colour-blind mode (alternative VFX colours).
Demo video: https://drive.google.com/drive/folders/1_h4WgMM67InN58y7YR7RUYqJjCG2CmOJ?usp=sharing

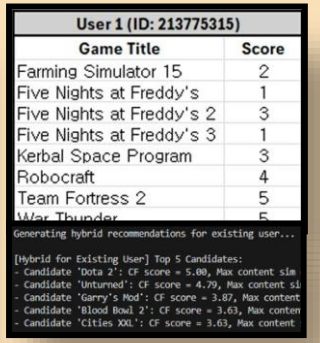
‹ Steam Game Recommender System ›

Language: Python
With over 100,000 titles on Steam, finding games that suit a player is hard. Using a Steam review dataset from Kaggle, cleaned the data, analysed user preferences and built two recommendation models: a collaborative filtering model that uses K-Nearest Neighbours to find similar users, and a neural network that uses game tags as features. The neural network achieved higher novelty, recommending a more diverse range of games.





	A	B	C	D	E	F
1	user_id	game_title	status_pla	total_playt	score	game_id
20406	69856757	DogFighter	TRUE	0,3	1	42500
20407	69856757	Dota 2	TRUE	1377	5	570
20408	69856757	Dungeon D	TRUE	11,7	4	65800
20409	69856757	Frozen Sy	TRUE	2,7	2	96200
20410	69856757	HOARD	TRUE	1,9	2	63000
20411	69856757	Left 4 Dea	TRUE	38	4	550
20412	69856757	Mafia	TRUE	18,1	4	40990
20413	69856757	Max Payn	TRUE	3,9	2	12140
20414	69856757	Mount & B	TRUE	5,7	3	22100
20415	69856757	Nation Rec	TRUE	4,8	3	39800
20416	69856757	No More R	TRUE	0,6	1	224260
20417	69856757	PAYDAY 2	TRUE	98	5	218620
20418	69856757	Panzar	TRUE	0,4	1	240320
20419	69856757	Team Fortr	TRUE	25	4	440
20420	69856757	Vlad the Im	TRUE	4,5	3	295850
20421	69942754	Left 4 Dea	TRUE	98	4	550
20422	69942754	Mafia II	TRUE	112	5	50130



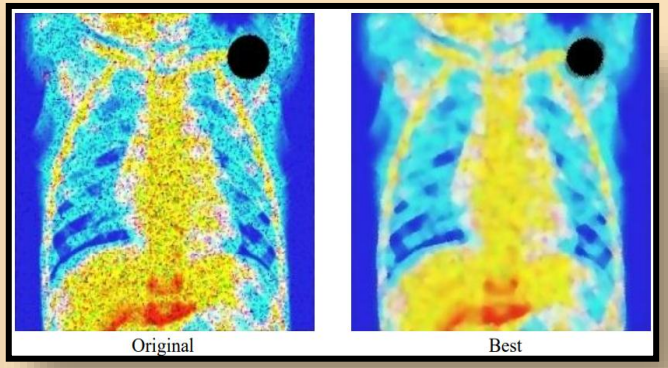
User 1 (ID: 213775315)	
Game Title	Score
Farming Simulator 15	2
Five Nights at Freddy's	1
Five Nights at Freddy's 2	3
Five Nights at Freddy's 3	1
Kerbal Space Program	3
Robocraft	4
Team Fortress 2	5
War Thunder	5

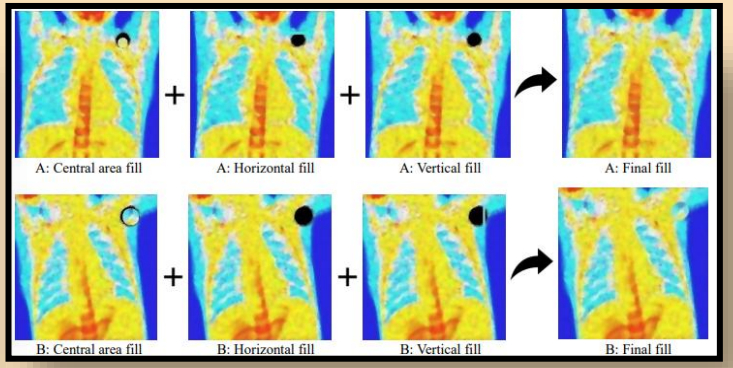
Generating hybrid recommendations for existing user...
[Hybrid for Existing User] Top 5 Candidates:
- Candidate 'Dota 2': CF score = 5.00, Max content sim
- Candidate 'Unturned': CF score = 4.79, Max content si
- Candidate 'Garry's Mod': CF score = 3.87, Max content
- Candidate 'Blood Bowl 2': CF score = 3.63, Max conten
- Candidate 'Cities XXL': CF score = 3.63, Max content

‹ X-ray Image Restoration ›

Language: Python (OpenCV)

Designed an X-ray image restoration algorithm to support a tuberculosis diagnostic device. Removed noise by combining OpenCV blurring with a custom colour-filtering algorithm, then built a filling algorithm that restores missing regions naturally by combining central, horizontal and vertical fills based on body symmetry and surrounding pixel data, and automated the full pipeline. Through continuous refinement, the system achieved 95% diagnostic accuracy in testing.

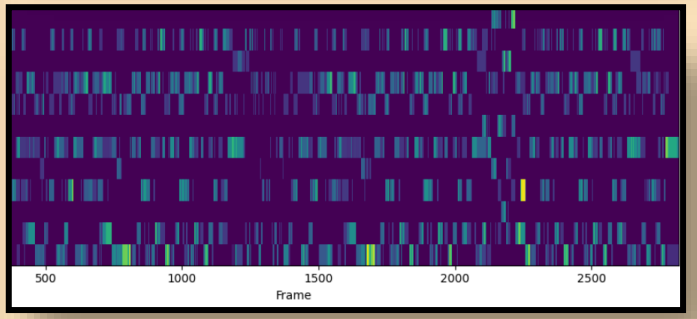




‹ Automatic Ringtone Generator ›

Language: Python (Pretty MIDI)

Developed a tool that takes a music file and automatically produces an MP3 ringtone of around 30 seconds. The track is split into small segments, and an algorithm detects repeated patterns to identify and extract the chorus, the most recognisable part of a song. To create a smooth transition from the iconic iPhone marimba opening, applied a marimba sound font and matched the tempo, then merged the opening with the processed chorus.



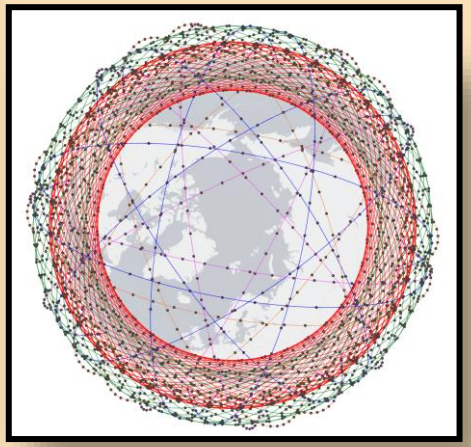


Demo video: https://drive.google.com/drive/folders/1Lz11Rt41v6SWbCIfxail2tpE_isIQwPf?usp=sharing

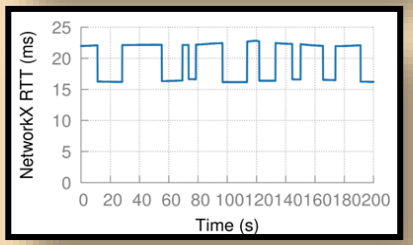
‹ LEO Network Topology Optimisation ›

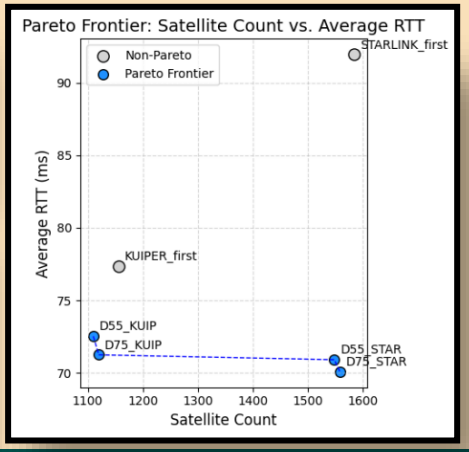
Language: Python on Linux (Hypatia)

Integrated the Hypatia framework with ns-3 to build an environment that measures round-trip time (RTT) across simulated TCP flows. By simulating a range of LEO satellite constellation models, including Starlink- and Kuiper-based configurations, validated how topology parameters affect network performance. A Pareto frontier analysis of satellite count against average RTT showed that efficient response times can be achieved with fewer satellites. The project built experience in tackling mathematically complex problems systematically through simulation and data.





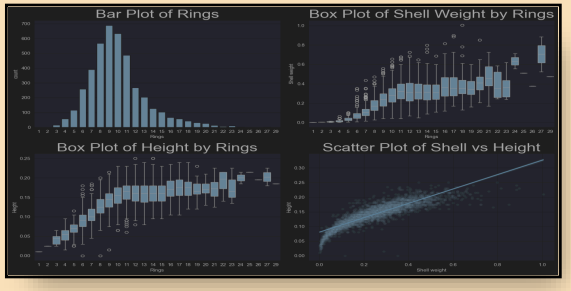




‹ Abalone Age Prediction (Machine Learning) ›

Language: Python

Built a model to predict the age of abalone, measured by the number of shell rings, from morphological features such as length, height, weight and shell thickness. After preprocessing the data, applied regression-based machine learning, compared several algorithms to select the best-performing model and improved its accuracy through hyperparameter tuning.

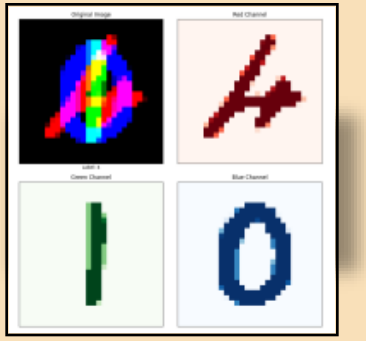


‹ Image Classification & Generation (Deep Learning) ›

Language: Python (PyTorch)

Discriminative model: Built a classifier for AddNIST (28×28 images, 20 classes) based on the LeNet-5 architecture. To reduce errors caused by overlapping colours, the network processes the RGB channels independently. Training each channel separately was also tested, but it reduced efficiency.

Generative model: Built an image generation model for CIFAR-100 using a StyleGAN architecture.

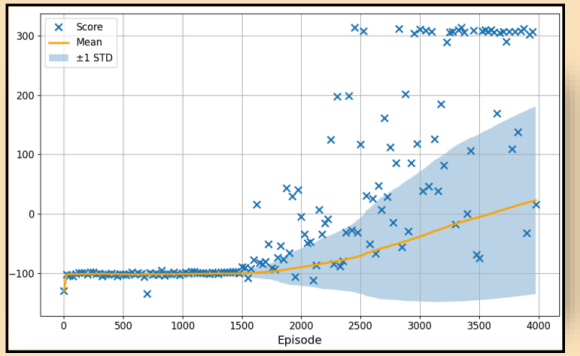




‹ Bipedal Walking Agent (Reinforcement Learning) ›

Language: Python

Trained a TD3 agent to walk across uneven terrain in the BipedalWalker environment, using Q-learning with delayed policy updates. Analysis revealed that the default reward was dominated by the fall penalty (-100), which led the agent to learn an overly cautious policy. Addressed this with custom reward functions such as a dynamic fall penalty, and boosted exploration noise early in training before gradually reducing it. Scores improved steadily over 4,000 episodes, while high resource use highlighted the need for a more efficient training structure.



Reports: https://drive.google.com/drive/folders/1xmyw2BGPJ87hG4jfb8olPvam9YPZF2J7?usp=drive_link